

Here is a step by step guide on the redevelopment of the Ecomute Webapp:

Step 1 - Redesign and Research:

The first stage of our redevelopment was making sure that we could implement the design we were given by researching potential methods of doing so. We started by researching better react libraries to use, as we thought that our implementation of the webapp in pure react could be improved upon. This resulted in us finding a library called Material-UI, or MUI for short. It provided us with a robust and flexible toolkit to build a visually appealing and highly functional user interface. Here are some of the main reasons why we picked this library:

- **Comprehensive** - Contains a wide range of ready-to-use components that cover many UI needs, from simple buttons to complex data tables. This allows us to be more creative and implement elements we wouldn't be able to do otherwise-

- **Modern Design** - It adheres to Google's Material Design principles, ensuring a modern, clean, and consistent design across all the different components. This also significantly speeds up the development process, as developers can trust that the components will work well together visually right out of the box.

- **Performance and Optimization** - MUI components are designed with performance in mind. The library supports tree-shaking, which means that only the parts of the library that are used are included in the final product. This results in faster load times and provides smoother interactions and transitions, enhancing the feeling of responsiveness for the users.

Using this new library, the Splash Page, the Login Page and the Sign Up page were all updated using the MUI library, using inbuilt components like the buttons, text boxes and popups. Then the basic functionality was added, only allowing a user with correct details to log into the app, and if they were to enter their details wrong, then appropriate error messages would pop up to inform the user what has gone wrong. To add further functionality to these pages, we made sure that there was a form of user input cleaning, making sure that the user could only enter in the expected text and nothing unexpected. These new designs have made the app look more 'official' and modern, allowing for a better overall user experience.

Now these next steps are the steps taken to redevelop the main dashboard:

Step 2 - Basic Map Integration and Functionality:

- We first started by implementing the basic elements of the page, being the header, the footer and how the main section was going to be laid out. We also changed the header so that it contained less buttons navigating to pages, because due to the new requirements, some pages, like the AI Page and Data Manager Page became redundant. Moving on to the main section between the header and the footer, we decided to split it into three sections, a section for the interactive map, a section for the user to input their information and another to produce the graphs and statistics from the users data. For the interactive map, we decided to use Leaflet for these reasons:

- **Simplicity** - The Leaflet API makes it very easy to implement basic map functionalities like displaying a map, adding markers, and drawing routes. This simplicity allowed for quick development and iteration, which we need to be doing as we are under time constraints.

- **Performance** - It makes sure maps load quickly and run smoothly on all devices. This performance is critical for our webapp, as users expect real-time interactions without any lag.

- **Extensibility** - The library supports numerous plugins, allowing for developers to add complex functionalities to their maps without significant overhead. For our webapp, this meant the ability to incorporate routing, geocoding, and other advanced features as the application evolved.

We used the MapContainer and Marker components from react-leaflet, which allowed us to implement location selection in two ways, when the user places a marker on the map OR by typing in their postcode and displaying a predefined destination - in this case being the Eviden headquarters in central London. A challenge we experienced was getting the marker icons to appear correctly, so there is only one placed marker by the user on the screen and not multiple, which we solved by constantly updating the map with the new information.

Once we got the map and the user input working, we created some calculators which gave us a distance between the point provided by the user and the end point and an estimated time based on the mode of transport being used, which consisted of: walking, cycling, public transport, petrol, electric and diesel cars. We have these displays currently on the website in the centre section below the area the user will input their postcode, so we can test if what we are doing is working. We then added some boxes on the third section of the webapp, just as a preliminary visual on where the graphs will be going.

Step 3 - Advanced Map Interactions, Input Validation and Geographic Restrictions:

Once the basic components of the page and of the map were implemented, we could now focus on the more advanced functions of the map, which were restricting the user to picking only a specific area in the map and to implement some sort of visual route drawing going from a start position to the predefined end point.

To implement this route drawing we used the OSRM backend for routing, bettering the LocationPicker for map clicks, and handling postcode search for start position setting. The OSRM is a routing engine designed to provide fast and accurate route planning services for road networks. Utilising data from OpenStreetMap (OSM), OSRM calculates the shortest path between two or more points and delivers detailed instructions and information essential for navigation. A challenge we faced when doing this was making sure that only one route showed up on the map at any given point, as the calculateRoute functions would produce the routes but not remove them.

After doing this, we thought that we should restrict the user from picking any random postcode or point on the map as this could cause issues in the database, so we restricted user input to valid London postcodes and prevented marker placement outside the M25 boundaries, as it would be unrealistic for people to work in Central London to live outside the M25. How we did

this was by using regex patterns for postcode validation and added checks against a simplified M25 bounding box. By completing this, we have created a comprehensive yet simple validation mechanism for London postcodes and accurately representing the M25 boundary.

Step 4 - Carbon Emissions Visualization

Now to another key part of our project, the Carbon Emission calculator and visualisation. The UI team worked together with the AI team, making sure the calculators are made to produce accurate results. Once the calculators were made, they were sent to the UI where we implemented them into our code - the calculators were made in javascript so they were very easy to integrate into the front-end React code. We had to edit them slightly so that they updated based on the distance between the start and end pointers, but that was it.

What the calculator does is take distance in metres and transportation mode to calculate CO2 emissions. For walking, cycling, and buses, it multiplies the distance by emissions per kilometre. For petrol and diesel cars, it calculates fuel consumed based on distance and fuel efficiency, then multiplies by CO2 per litre. For electric cars, it calculates energy consumed based on distance and energy efficiency, then multiplies by emissions per kWh. The result is returned as a string with two decimal places.

Now to display the different graphs we used a library called ReCharts, as we wanted to implement radial (doughnut), bar, and line charts to show the emissions data. ReCharts is a composable charting library built on React components, offering a straightforward way to integrate charts into React applications, using the power of D3.js under the hood but abstracting away its complexity. We chose to use Recharts because it is:

- **Easy of Use** - It simplifies the process of adding charts to a React application, allowing developers to work with familiar JSX syntax and React's component lifecycle
- **Reactive and Dynamic** - Because it is built for React, Recharts charts are reactive and update dynamically with state or prop changes. This responsiveness is key for the Ecomute application, where emissions data can change based on user inputs like commuting mode or distance
- **Composability** - Recharts adopts a component-based approach to building charts, allowing developers to compose charts with components such as <Line>, <Bar>, <Area>, etc. This composability made it easier to experiment with different types of visualisations and to combine them in innovative ways to present emissions data effectively

Step 5 - Navigation and Interactivity

We've kept the base smooth navigation between different parts of the webapp that we had before, as we think this is key to the users experience. We did this before using React Router for page transitions, interactive login and loading pages, and dynamic updates based on user selections.

Step 6 - User Validation and database integration:

We have also worked together with the backend Team and have connected the website to the database so we now have a way to safely secure users passwords by hashing them and storing

them in a separate location and not in the front end code - documentation on the actual creation of this can be found on the backend team branch

Step 7 - User Customizability

We have created an account settings page which will allow you to alter details about yourself, journey and other aspects to ensure your information is up to date. You are able to change your full name, email, phone number, username and bio. We will be implementing separate pages for notifications, appearance, privacy and security, and help and support. This should give the user a lot of flexibility in terms of how they want to use the app, e.g. If they want to be notified for each potential carpool opportunity or be notified once per month (within notifications). Appearance would have some accessibility options such as dark and light mode - with the possibility of various font sizes. Help and support would have some FAQs along with some contact details for customer support.

Step 8 - User feedback and Improvement:

As a team, after the first iteration of the UI was implemented, we got together and analysed the new UI design. The main feedback we got was that it was too cluttered and so we updated the section of a user interface designed for selecting a mode of transport and entering a starting postcode, making the map bigger as well. The updates focus on improving the form's aesthetics, usability, and overall alignment with modern web design principles. The improvements included adjusting to the layout, spacing, visual elements, and interactive feedback to create a more engaging and user-friendly experience. Here they are in more detail:

- Layout and Structure:

- The form is enclosed in a Box component that serves as the main container, ensuring all child components are aligned and organised. This container is set to take up 50% of its parent container's width and is flexibly designed to adapt to various screen sizes while maintaining its structural integrity.

- Styling Improvements:

- **Background and Borders** - The form's background is set to match the application's theme for a seamless integration. Rounded corners and a shadow effect are applied for a soft, more modern look.

- **Typography** - The headings, so all of the components included in Typography tags, are styled to stand out, with increased bottom margin for better separation from other elements. Text is centrally aligned for consistency as well.

- **Input Fields and Buttons** - The postcode input field (TextField) utilises the 'filled' variant for a modern, minimalistic appearance. Custom styling is applied to allow the background colour to change on hover and focus, enhancing user interaction. The search button (IconButton) is styled to visually respond to user actions, with theme-consistent hover effects.

- Mode of Transport Selection:

- Each mode of transport is represented by an icon, with the current selection highlighted, providing immediate visual feedback to the user. They are also arranged in the centre and evenly spaced.

- More Responsive Design:

- The form's design considers responsiveness, with flex properties and width percentages ensuring it adapts well to different screen sizes. Spacing and layout adjustments make sure that the form remains aesthetically pleasing and functional across devices.

- Step 9 - Connecting the Frontend with the backend:

In the development of our application, connecting the React frontend to the Supabase backend was an important step that enabled dynamic data interactions and real-time updates. The process began with setting up a Supabase project, where a new PostgreSQL database was created, and essential configurations, including API keys and security settings, were established.

Following the backend setup, the Supabase JavaScript client was integrated into the React project, facilitating secure and efficient communication between the frontend and backend. This integration required the installation of the `@supabase/supabase-js` package and the creation of a `supabaseClient.js` file to initialise the Supabase client with our project-specific URL and API keys.

Data fetching was implemented within React components using the `useEffect` hook to make asynchronous calls to the Supabase database, retrieve journey information, and store it in the component state. This approach allowed for the dynamic rendering of journey data within the application, offering users real-time insights and interactions with the stored information.

- Step 10 - Design and implementation Of 'Previous Journeys' Page:

We have created a Calendar page to display all previous journeys in a clear and coherent format. We have included functionality which allows for the querying of postcodes, selecting the method of transport and the start and end date of the data entered. Initially in our prototype, we had decided to only include a single date to find data for, however we quickly realised this may be quite inefficient for the user if they would like to look at a period of time instead.

To build this page, we connected to the Journey Table in our supabase database, and displayed the following columns: `journeyid` (primary key), `postcode`, `estimateTimeTaken`, `distance`, `carbonEmissions` and `methodOfTransport`. We felt these were the most important pieces of information to be displayed to the user so that's what we have shown here. We have used the table formatting as part of the MUI library

- Step 11 - Finalisation of the Dashboard:

Firstly, we started by implementing radial bar charts to visually represent carbon emissions, ensuring that the graphs dynamically adjust according to the emissions level. This feature helps users visually see their carbon footprint in relation to different transport modes and potential carbon emissions. Based on the selected mode of transport, our system provides personalised recommendations encouraging users to opt for greener alternatives. This feature aims to promote environmentally friendly travel habits.

We then edited the dashboard code and the database scheme and rules set up so that it could save and accept certain elements the results produced from the user, these being mode of transport, postcode, address, calculated emissions and the corresponding vehicleID. For example, we had to edit the rules of the database to allow for NULL values in the vehicleID column, as some methods of transport do not have a vehicleID, e.g. walking, cycling and public transport.

Some challenges we faced:

- Complexity of Multiple Emissions Calculations - The diversity of factors influencing carbon emissions made the calculation algorithm complex, however we made sure that these were as accurate as possible while accommodating different modes of transport required thorough research and testing.
- Graphical Representation Limitations - Dynamically adjusting the radial bar charts to accurately depict emissions levels was challenging but we overcame this by taking the largest carbon emission value that was produced out of the 5 modes of transport and made dynamically changed the max number the graphs went up to based on that so that they were all being measured on the same scale.

Next, we edited the way we validated and extracted London and Greater London postcodes within the webapp. The process involved using a better regular expression to match user-inputted or selected postcodes against specific patterns that correspond to those regions. This functionality is critical for our webapp as it requires users to input a location within the Greater London area, ensuring the data's geographical relevance and accuracy.

This enhanced pattern includes additional postcode areas relevant to Greater London (BR, CR, DA, EN, HA, IG, KT, RM, SM, TW, UB, WD), meaning it provided broader coverage. However, it's important to note that some of these areas might extend beyond Greater London's official boundaries. The regex patterns were utilised in functions responsible for validating and extracting postcodes from user inputs or map selections:

- Validating User Input - Ensuring that the user-entered postcodes match the defined regex patterns, thus confirming they fall within London or Greater London.
- Extracting Postcodes from Addresses - Using the regex to extract postcode portions from full addresses returned by geocoding services, facilitating accurate location processing and storage

Through careful regex design and integration into key application functionalities, we successfully implemented a system for validating and extracting London and Greater London postcodes, enhancing the application's geographic data accuracy, ensuring that location-based user inputs are within the desired area.

Next, we included a small popup that appears on the bottom of the screen when the user wants to save their journey to the database, giving them a visual cue that this was successful. We then fixed the graphs in the 'User Results' section, as it now shows the users carbon emission results.

We then integrated the graphs given to us by the MLM team which are used to visually represent user data related to journeys, such as shared journeys, carbon emissions, and distance travelled. This integration aimed to provide users with insights into their commuting patterns and environmental impact. We also modernised the display of journey information in the dashboard to improve clarity and aesthetics. Due to limitations in dynamically applying gradients without direct DOM manipulation in React, we focused on improving the chart through configurable options such as colours and animations.

Error Handling and User Feedback was next and error handling was improved across the dashboard to provide users with clear, actionable feedback in case of issues such as duplicate data entries. A success dialog was implemented to confirm the successful submission of journey details.

- Step 12 - Finalisation of the Log-in and Sign Up Page:

Sign Up Page - Here are the modifications and enhancements made to the Sign-Up page for the application which improve the user experience, data validation, and integration with the database for user registration. Here are the implemented Features and Changes:

- Input Validation Enhancements:

- **Name Fields Validation** - First and Last Name fields now accept only letters and hyphens. This was achieved by using a regular expression to filter the input.
- **Department Selection** - Converted the Department field from a text input to a dropdown selection. This ensures users select from predefined department options, eliminating input errors and standardising data.
- **Email Validation** - Added validation to ensure that the email field contains an "@" symbol, following standard email format requirements.
- **Phone Number Validation** - The Phone Number field now only accepts numbers and a plus symbol, with a limit of 11 digits to ensure a valid phone number is entered.
- **Password Requirements** - Enforced password rules requiring a minimum of 8 characters, including at least one number and one special character. Implemented visual feedback for users to correct inputs that do not meet these criteria.
- **Terms of Agreement** - Implemented a checkbox for users to agree to the Terms of Use before signing up. The Terms of Use can be viewed in a popup dialog by clicking a link. Signing up is disabled until the terms are accepted.
- **Success Dialog Box** - Upon successful registration, a dialog box now appears informing the user that their details have been saved successfully. This dialog includes a button that navigates the user to the login page.
- **Database Integration**
 - User details are now saved to a database upon successful validation of the sign-up form. The information saved includes the user's name, department (converted to a department ID for database use), email, phone number, and password.

- Added error handling to inform users if an account with the provided email already exists or if there was an issue saving their details.

- Code Structure and Logic Adjustments:

- Removed the "Confirm Password" field and related validation logic to streamline the sign-up process.

- Adjusted the handleSignUpSubmit function to perform client-side validation before attempting to save user data to the database.

Implemented a conditional display of the success dialog upon successful user registration, with a redirection option to the login page.

- Miscellaneous UI Improvements:

- Made slight adjustments to UI elements (e.g., button colours, input field rounding) for a more cohesive look and feel.

Testing for the Sign Up Page:

1) Name Fields (First Name, Last Name) - Type: Text - Tests:

- Accepts letters and hyphens - It does accept these symbols
- Rejects numbers and special characters - It does reject these characters
- Tests for empty input - returns a popup message that says to fill in this fields

2) Department - Type: Dropdown Selection - Tests:

- Selection from predefined options - Allows the selection of 1 of the predefined options
- Tests for no selection (if field is required) - returns a popup message that says to fill in this fields

3) Email - Type: Text (with email format validation) - Tests:

- Accepts valid email formats (contains "@") - It does accept these symbols
- Rejects invalid email formats - Doesn't allow the user to submit this text without it having a @ in it
- Tests for empty input - returns a popup message that says to fill in this fields

4) Phone Number - Type: Numeric - Tests:

- Accepts numbers and a plus symbol - It does accept these symbols
- Limits input to 11 digits - Doesn't allow the user to input more than 11 digits
- Rejects letters and special characters - Doesn't allow the user to input these characters
- Tests for empty input - returns a popup message that says to fill in this fields

5) Password - Type: Text (with specific requirements) - Tests:

- Min of 8 characters - doesn't allow submission of password with less characters
- Requires at least one number and one special character - doesn't allow submission of password with no letters or special characters.
- Rejects passwords that don't meet the criteria - returns a popup message that says to fill in this fields
- Tests for empty input - returns a popup message that says to fill in this fields

6) Terms of Agreement Checkbox - Type: Checkbox - Tests:

- Checks if the checkbox is selected - Fills in the ticked box to visually show
- Tests for submission attempts without agreement - Does not allow the user to press the sign in button without the checkbox being ticked.

Step 13 - Similarity Search MLM integration with the UI:

The goal of this step is to enable a "Potential Matches" feature within a React dashboard that interacts with the Flask API to fetch and display user data based on similarity metrics.

System Architecture

Frontend (React Dashboard)

- Provides interactive components for user engagement.
- Sends requests to the Flask API to retrieve similarity-based user data.
- Displays the results in a structured and user-friendly format.

Backend (Flask API)

- Handles requests from the frontend.
- Executes similarity search logic against a database.
- Returns relevant user data in response to queries.

Database

- Stores user data and other relevant information.
- Accessed by the Flask API to perform data retrieval based on search criteria.

Flask API Setup

Flask Application Initialisation:

- A Flask server is set up to handle API requests. It is responsible for fetching data from the database based on the similarity search algorithm.

API Endpoints:

- A primary endpoint `/api/search` accepts POST requests with search parameters and user ID. It processes these requests to return a list of potential matches based on predefined similarity criteria.

Database Interaction:

- Flask interacts with a database using an ORM or direct database queries. It retrieves user data that matches or closely aligns with the search parameters sent from the frontend.

React UI Integration

Components:

- **Map Display** - Utilises Leaflet to show interactive maps where users' locations and routes are visualised.
- **Search Component** - Allows users to input search criteria that are sent to the Flask API.

- **Results Display** - Shows potential matches retrieved from the Flask API in a user-friendly format, including cards and interactive maps.

State Management:

- Uses React state hooks (useState) to manage local state and useEffect or side effects such as API calls on component mount or update.

API Interaction:

- React components use the fetch API to make POST requests to the Flask backend. The responses (potential matches) are then rendered in the UI.

Event Handlers:

- Define functions to handle user actions like clicking on the "Find Potential Matches" button, which triggers the API call.

Styling:

- Material-UI components are used extensively for layout and styling to ensure a consistent and responsive design.

Error Handling and User Feedback

- **Error Handling** - Both the Flask API and React frontend include error handling mechanisms to deal with network errors, bad responses, and other unexpected issues.
- **User Feedback** - Snackbar notifications and loading indicators inform the user about the system status (e.g., fetching data, success, or failure).

Security Considerations

- **API Security** - Flask API uses appropriate security measures such as CORS, authentication (e.g., JWT tokens), and input validation to protect against common vulnerabilities.
- **Data Privacy** - Ensures sensitive user data is handled securely, implementing data encryption and secure access protocols.

Deployment

- **Flask API** - Deployed on a cloud service like AWS Elastic Beanstalk or Heroku.
- **React Application** - Hosted on platforms such as Netlify or Vercel that support SPA (Single Page Application) architectures.
- **Continuous Integration/Deployment** - Set up CI/CD pipelines using tools like GitHub Actions to automate the deployment process on commits/merges to the main branch.